



Operadores Lógicos

Python Básico | ⌚ 16 minutos

Operadores Lógicos

En esta lección vamos a aprender sobre un nuevo tipo de operadores, los **operadores lógicos**. Estos son operadores que nos permiten hacer preguntas a la computadora, para las cuales siempre vamos a obtener una respuesta binaria:

True o **False**.

Este concepto va a ser la base para llevar nuestros programas al siguiente nivel, ya que vamos a poder tomar decisiones en base a ciertas condiciones.

Igual a **==**

Este primer operador revisa si dos valores son **idénticos**, tomando en cuenta su valor y su tipo de dato. Es importante no confundirlo con el operador de asignación (**=**).

```
1 print(10 == 10)
2
3 user_password = "Cheese123"
4
5 print(user_password == "Cheese123")
6 print(user_password == "NotCheese123")
```

Copiar

✓ True
True
False

Distinto a `!=`

Este operador revisa si dos valores son **diferentes**. Es el opuesto del operador Igual a `==`. Retorna `True` si NO son iguales.

```
1 print("Hola" != "Mundo")
2
3 value_1 = 25
4 value_2 = 25
5
6 print(value_1 != value_2)
```

[Copiar](#)

✓ True
False

Mayor a `>`

Revisa si un número es **mayor** a otro. Al igual que como lo aprendimos en matemática básica, la apertura de la flecha apunta hacia el valor mayor para que el resultado sea `True`.

💡 **Nota importante:** cuando ambos valores son iguales retorna `False`. Esto es porque el operador `>` requiere que el valor de la izquierda sea estrictamente mayor para que sea `True`.

```
1 grade_1 = 95
2 grade_2 = 70
3
4 print(grade_1 > 70)
5 print(grade_2 > 70)
6 print(grade_2 > grade_1)
```

[Copiar](#)

✓ True
False
False

Mayor o igual a `>=`

Revisa si un número es **mayor** o **igual** a otro. Muy similar al anterior, pero en este caso si pueden ser iguales y por ende retorna `True`. En que este caso, cuando ambos valores son iguales retorna `True`.

```
1 grade_1 = 95
2 grade_2 = 70
3
4 print(grade_1 >= 70)
5 print(grade_2 >= 70)
6 print(grade_2 >= grade_1)
```

[Copiar](#)

✓ True
True
False

Menor a `<`

Revisa si un número es **menor** a otro. La punta de la flecha apunta al valor que esperamos que sea el más pequeño.

💡 **Nota importante:** cuando ambos valores son iguales retorna `False`

```
1 account_balance = 50
2 item_price = 100
3
4 print(account_balance < 100)
5 print(item_price < 100)
6 print(item_price < account_balance)
```

[Copiar](#)

✓ True
False
False

Menor o igual a <=

Revisa si un número es **menor** o **igual** a otro. Muy similar al anterior, pero en este caso si pueden ser iguales y por ende retorna `True`.

```
1 account_balance = 50
2 item_price = 100
3
4 print(account_balance <= 100)
5 print(item_price <= 100)
6 print(item_price <= account_balance)
```

[Copiar](#)

✓ True
True
False

Negación not

Invierte un valor a su **opuesto**, este es usado principalmente en `booleans`. Si el valor es `True` pasa a `False` y viceversa.

```
1 # Invertir un booleano
2 print(not True)
3 print(not False)
4
5 # Invertir el valor de una variable
6 is_adult = True
7 print(not is_adult)
8
9 # Invertir el resultado de una operación lógica
10 print(not (5 > 10))
```

[Copiar](#)

✓ False
True
False
True

Resumen

Operación	Símbolo	Descripción	Ejemplos
Igual a	<code>==</code>	Revisa si dos valores son idénticos.	<code>10 == 10 → True</code> <code>10 == "10" → False</code>
Distinto a	<code>!=</code>	Revisa si dos valores son diferentes.	<code>10 != 10 → False</code> <code>"Key" != "Lock" → True</code>
Mayor a	<code>></code>	Revisa si un número es mayor a otro.	<code>90 > 40 → True</code> <code>50 > 50 → False</code>
Mayor o igual a	<code>>=</code>	Revisa si un número es mayor o igual a otro.	<code>90 >= 40 → True</code> <code>50 >= 50 → True</code>
Menor a	<code><</code>	Revisa si un número es menor a otro.	<code>90 < 40 → False</code> <code>50 < 50 → False</code>
Menor o igual a	<code><=</code>	Revisa si un número es menor o igual a otro.	<code>90 <= 40 → False</code> <code>50 <= 50 → True</code>
Negación	<code>not</code>	Invierte el valor de un booleano.	<code>not True → False</code> <code>not False → True</code> <code>not (5 > 10) → True</code>

Combinando Condiciones: **and** y **or**

A medida que avancemos en nuestro camino, nos encontraremos con situaciones donde una sola pregunta no es suficiente. Para abordar escenarios más complejos, necesitamos la capacidad de unir varias condiciones lógicas. Aquí es donde entran en juego los operadores **and** y **or**.

Operador **and**

Este operador analiza si **todas** las condiciones unidas son **True**. Si al menos una de ellas es **False**, el resultado final será **False**. Es muy estricto, todo debe cumplirse.

```
1 is_weekend = True
2 is_sunny = True
3
4 # Ambas condiciones se cumplen
5 print(is_weekend and is_sunny)
6
7 is_weekend = True
8 is_sunny = False
9
10 # Una condición falla
11 print(is_weekend and is_sunny)
```

[Copiar](#)

✓ True
False

Operador **or**

Este operador es más flexible, analiza si **al menos una** de las condiciones es **True**. El único caso en el que devuelve **False** es cuando **todas** las condiciones son **False**.

```
1 has_coffee = True
2 has_tea = False
3
4 # Al menos una es verdadera
5 print(has_coffee or has_tea)
6
7 has_coffee = False
8 has_tea = False
9
10 # Ninguna es verdadera
11 print(has_coffee or has_tea)
```

Copiar

✓ True
False

Prioridad con Paréntesis ()

En matemáticas, sabemos que la multiplicación se resuelve antes que la suma. De manera similar, en programación los operadores lógicos tienen un orden de prioridad, se evalúa primero el `and` y luego el `or`.

Pero nosotros podemos usar paréntesis `()` para agrupar condiciones y asegurar que la lógica sea la correcta.



Pensemos en un sistema de entrada a una fiesta. Para entrar, una persona necesita:

Tener su entrada `Y` además, cumplir con una de dos condiciones:
ser mayor de edad `O` tener permiso de sus padres.

Si no usamos paréntesis, el resultado puede ser el incorrecto. Veamos el caso sin paréntesis:

```
1 has_ticket = False
2 is_adult = True
3 has_permission = True
4
5 # (False and True) or True -> False or True -> True
6 print(has_ticket and is_adult or has_permission)
```

Copiar

 True

Por el orden de los operadores se ejecuta primero el `and`, por ende ¡tenemos un error! En cambio, si implementamos los paréntesis, resolvemos el problema:

```
1 has_ticket = False
2 is_adult = True
3 has_permission = True
4
5 # Forzamos a evaluar primero el grupo (edad o permiso)
6 # False and (True or True) -> False and True -> False
7 print(has_ticket and (is_adult or has_permission))
```

Copiar

 False